



Seminar Theoretische Informatik

Berechenbarkeit

Kai Höfig



- ◆ Der Begriff der „Berechenbarkeit“
- ◆ Turingmaschinen
- ◆ LOOP, WHILE und GOTO Programme
- ◆ Primitiv rekursive Funktionen
- ◆ Churchsche These
- ◆ Halteproblem, Unentscheidbarkeit, Reduzierbarkeit
- ◆ Postsches Korrespondenzproblem
- ◆ Unentscheidbare Grammatiken
- ◆ Satz von Gödel



Intuitive Berechenbarkeit

- ♦ Eine Funktion $f : \mathbb{N}^k \rightarrow \mathbb{N}$ ist **berechenbar**, wenn es einen Rechenverfahren gibt, das f in endlicher Zeit berechnet.
- ♦ Ist f eine an manchen Stellen undefiniert, also eine **partielle** Funktion, soll der Algorithmus bei entsprechender Eingabe nicht stoppen.

- ♦ Sind die folgenden Funktionen berechenbar?

$$a(n) = x^3$$

$$b(n) = \sqrt{x}$$

$$f_{\pi}(n) = \begin{cases} 1 & \text{falls } n \text{ ein Anfang von } \pi \\ 0 & \text{sonst} \end{cases}$$

$$d(n) = \begin{cases} 1 & \text{falls } n \text{ irgendwo in } \pi \\ 0 & \text{sonst} \end{cases}$$

$$e(n) = \begin{cases} 1 & \text{falls in } \pi \text{ mindestens } n\text{-mal hintereinander die 7 vorkommt} \\ 0 & \text{sonst} \end{cases}$$

$$f(n) = \begin{cases} f(3n+1) & \text{wenn } n \text{ ungerade und nicht } 1 \\ f(n/2) & \text{wenn } n \text{ gerade} \\ 1 & \text{wenn } n = 1 \end{cases}$$



Berechenbarkeit von $\mathbb{R}$

- ♦ So wie $f_\pi(n)$ eine Funktion ist, der die Zahl π zugeordnet werden kann, gibt es auch eine Funktion $f_e(n)$, die der Eulerschen Zahl zugeordnet werden kann. Solche Funktionen existieren nicht für alle reellen Zahlen.
- ♦ Denn: Es existieren überabzählbar viele reelle Zahlen, aber nur abzählbar viele Berechnungsverfahren. Überabzählbar heißt eine Menge, die nicht endlich ist und zu der es keine Bijektion in die Menge der natürlichen Zahlen gibt. Berechnungsverfahren sind abzählbar, da sie durch endlichen Text beschreibbar sind.



Turingmaschine, **d**eterministisch und **n**icht**d**eterministisch

♦ $M = (Z, \Sigma, \Gamma, \delta, z_0, \square, E)$ mit

♦ Zustandsmenge Z

♦ Eingabealphabet Σ

♦ Arbeitsalphabet Γ

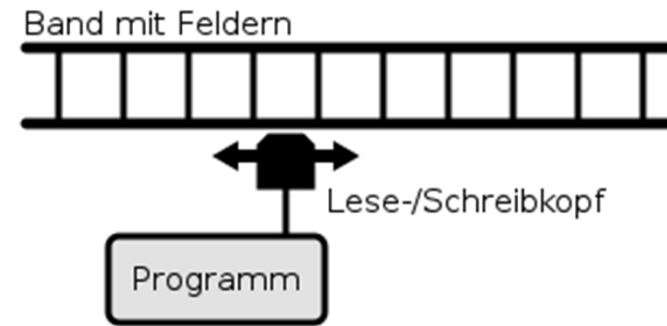
♦ Übergangsfunktion $\delta : Z \times \Gamma \rightarrow Z \times \Gamma \times \{Links, Rechts, Neutral\}$ **d**

$\delta : Z \times \Gamma \rightarrow \mathcal{P}(Z \times \Gamma \times \{Links, Rechts, Neutral\})$ **nd**

♦ Startzustand $z_0 \in Z$

♦ Blank $\square$

♦ Endzustände $E \subseteq Z$





Beispiel Turingmaschine n++

$$M = (Z, \Sigma, \Gamma, \delta, z_0, \square, E)$$

$$M = (\{z_0, z_1, z_2, z_e\}, \{0, 1\}, \{0, 1, \square\}, \delta, z_0, \{z_e\})$$

♦ mit

$$\delta(z_0, 0) = (z_0, 0, R)$$

$$\delta(z_0, 1) = (z_0, 1, R)$$

$$\delta(z_0, \square) = (z_1, \square, L) \quad \text{Ganz nach rechts}$$

$$\delta(z_1, 0) = (z_2, 1, L)$$

$$\delta(z_1, 1) = (z_1, 0, L) \quad \text{Addiere}$$

$$\delta(z_1, \square) = (z_e, 1, N)$$

$$\delta(z_2, 0) = (z_2, 0, L)$$

$$\delta(z_2, 1) = (z_2, 1, L) \quad \text{Wieder ganz nach links}$$

$$\delta(z_2, \square) = (z_e, \square, R)$$



Beispiel Turingmaschine n--

$$M = (Z, \Sigma, \Gamma, \delta, z_0, \square, E)$$

$$M = (\{z_0, z_1, z_2, z_e\}, \{0, 1\}, \{0, 1, \square\}, \delta, z_0, \{z_e\})$$

♦ mit

$$\delta(z_0, 0) = (z_0, 0, R)$$

$$\delta(z_0, 1) = (z_0, 1, R)$$

Ganz nach links

$$\delta(z_0, \square) = (z_1, \square, L)$$

$$\delta(z_1, 1) = (z_2, 0, L)$$

Es gibt eine 1, ziehe ab..

$$\delta(z_2, 0) = (z_2, 0, L)$$

..und gehe nach links

$$\delta(z_2, \square) = (z_e, 1, N)$$

$$\delta(z_2, 1) = (z_2, 1, L)$$

$$\delta(z_1, 0) = (z_3, 1, L)$$

sonst Übertrag

$$\delta(z_3, 1) = (z_2, 0, L)$$

$$\delta(z_3, 0) = (z_3, 1, L)$$

$$\delta(z_3, \square) = (z_e, \square, R)$$

Carry flag?

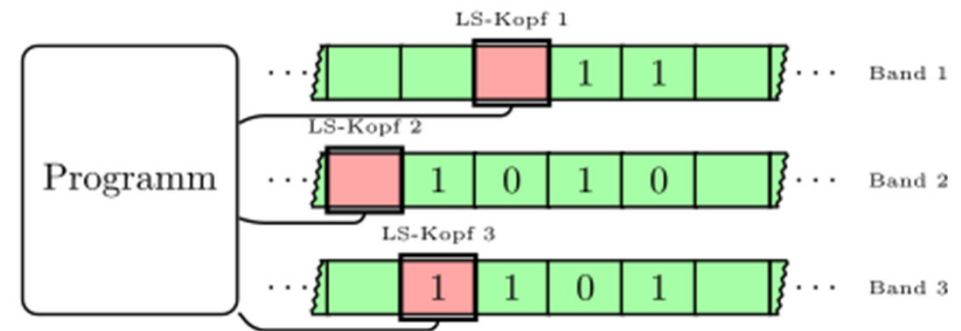


Mehr-Band Turingmaschinen

$$M = (Z, \Sigma, \Gamma, \delta, z_0, \square, E)$$

$$\delta: (Q \setminus F) \times \Gamma^k \rightarrow Q \times \Gamma^k \times \{L, N, R\}^k$$

- ♦ Zu jeder Mehrband-Turingmaschine gibt es eine Einband-Turingmaschine, die dieselbe Funktion berechnet.
- ♦ Kodiere dazu das Arbeitsalphabet neu als Potenzmenge der einzelnen Arbeitsalphabete und passe die Übergangsfunktion entsprechend an.



aktueller Zustand	geles. Symbol				schr. Symbol			neuer Zustand	Kopf-richtungen		
q_0	b	$\emptyset$	b	$\rightarrow$	b	$\emptyset$	b	q_0	N	R	N
q_0	b	1	b	$\rightarrow$	b	1	b	q_0	N	R	N
q_0	$\emptyset$	b	b	$\rightarrow$	$\emptyset$	b	b	q_0	R	N	N
q_0	$\emptyset$	$\emptyset$	b	$\rightarrow$	$\emptyset$	$\emptyset$	b	q_0	R	R	N
q_0	$\emptyset$	1	b	$\rightarrow$	$\emptyset$	1	b	q_0	R	R	N
q_0	1	b	b	$\rightarrow$	1	b	b	q_0	R	N	N
q_0	1	$\emptyset$	b	$\rightarrow$	1	$\emptyset$	b	q_0	R	R	N
q_0	1	1	b	$\rightarrow$	1	1	b	q_0	R	R	N
q_0	b	b	b	$\rightarrow$	b	b	b	q_1	L	L	N



Touring-Berechenbarkeit

- ♦ Eine Funktion $f : \mathbb{N}^k \rightarrow \mathbb{N}$ ist **Turing-berechenbar**, wenn es eine Turingmaschine gibt mit

$$f(n_1, \dots, n_k) = m$$

$$\Leftrightarrow$$

$$z_0 \text{ bin}(n_1) \# \dots \text{bin}(n_k) \# \vdash^* \square \dots \square z_e \text{ bin}(m) \square \dots \square$$

- ♦ Eine Funktion $f : \Sigma^k \rightarrow \Sigma$ ist **Turing-berechenbar**, wenn es eine Turingmaschine gibt mit

$$f(x) = y$$

$$\Leftrightarrow$$

$$z_0 x \# \vdash^* \square \dots \square z_e y \square \dots \square$$



Turingmaschinen Diskussion

- ◆ Wie kann man mit Turing Maschinen programmieren?
- ◆ Wie kann man ein Loop, While oder If gestalten?
- ◆ Wie kann man mehrere Parameter verarbeiten?



LOOP Programme

- ♦ LOOP Programme haben folgende Syntax

- ♦

Variablen	x0, x1, x2 ..
Konstanten	1, 2, 3, 4 ..
Trennsymbole	; :=
Operationszeichen	+ -
Schlüsselwörter	LOOP DO END

- ♦ Beispiel

```
y:=1
LOOP x DO y:=0 END;
LOOP y DO z=1 END;
```

Simuliert if (x=0) then z=1

- ♦ Dabei wird die LOOP Variable einmal ausgewertet. Änderungen im LOOP haben keine Auswirkung.

LOOP-berechenbar



WHILE-Programme

- ◆ WHILE Programme haben folgende Syntax

- ◆

Variablen	x0, x1, x2 ..
Konstanten	1, 2, 3, 4 ..
Trennsymbole	; :=
Operationszeichen	+ - != ==
Schlüsselwörter	WHILE DO END

- ◆ Beispiel

```
y:=x;  
WHILE  
    y != 0  
DO  
    y:=y-1;  
    z=z+1;  
END
```

*Simuliert LOOP x DO
z:=z+1; END*

WHILE-berechenbar



GOTO-Programme

- ♦ GOTO Programme haben folgende Syntax

♦ Variablen	x0, x1, x2 ..
Konstanten	1, 2, 3, 4 ..
Trennsymbole	; := :
Operationszeichen	+ - != ==
Schlüsselwörter	GOTO IF THEN HALT

- ♦ GOTO Programme bestehen aus Anweisungen A und Marken M

M1 : A1; M2 : A2; M3 : A3; ...

- ♦ Beispiel
- ```
M1 : IF x=0 THEN GOTO M2;
 z=z+1;
 GOTO M1;
M2 : ...
```

*Simuliert WHILE*  
*x!=0 DO z:=z+1*

GOTO-berechenbar



- ◆ Es gilt offensichtlich mindestens

$$\text{LOOP} \subset \text{WHILE} \subset \text{GOTO}$$

- ◆ Aber es ist

$$\text{LOOP} \subsetneq \text{WHILE} = \text{GOTO}$$

- ◆ Was müssen wir jetzt noch zeigen, damit wir verstehen, dass

$$\text{WHILE} = \text{GOTO}$$

- ◆ ..und wieso ist

$$\text{LOOP} \subsetneq \text{WHILE} \quad \text{bzw.} \quad \text{LOOP} \subsetneq \text{GOTO}$$



# Kleenesche Normalform für WHILE Programme

- ♦ Jede WHILE berechenbare Funktion kann durch ein WHILE Programm mit nur einer WHILE Schleife berechnet werden
- ♦ Beweis:
  - Sei P ein beliebiges WHILE Programm.
  - Wandle P in ein GOTO Programm um.
  - Wandle P wieder in ein WHILE Programm um.

- ♦ Skizze GOTO->WHILE:

```
count:=1;
WHILE count!=0 DO
 IF count=1 THEN A1 END;
 IF count=2 THEN A2 END;
 ...
 IF count=n THEN A3 END;
END;
```

*Marken    Anweisungen*



Es gibt eine Funktion die WHILE aber nicht LOOP-berechenbar ist

---

♦ Die Ackermannfunktion

$$A(m, n) = \begin{cases} n + 1 & \text{if } m = 0 \\ A(m - 1, 1) & \text{if } m > 0 \text{ and } n = 0 \\ A(m - 1, A(m, n - 1)) & \text{if } m > 0 \text{ and } n > 0 \end{cases}$$

- ♦ Ist eine praktisch nutzlose Funktion, mit der man aber beweisen kann, dass die durch kein LOOP-Programm berechenbar ist. Es gilt also

$$\text{LOOP} \subsetneq \text{WHILE}$$





# Mit GOTO lassen sich Turingmaschinen simulieren (1)

- ◆ Es sei P ein GOTO Programm mit
  - M1 : P1; Transformiert die Eingangsvariablen in Binärdarstellung
  - M2 : P2; Simuliert die Turingmaschine
  - M3 : P3; Erzeugt die Ausgabe

- ◆ P2 benutzt drei Variablen X, Y und Z, um das Band und den aktuellen Zustand zu speichern

$$\underbrace{a_{i_1} \dots a_{i_p}}_X \quad Z = l \quad \underbrace{a_{j_1} \dots a_{j_q}}_Y$$

- ◆ Allerdings werden die Daten nicht einfach so in X und Y gespeichert, da wir ja immer auf „das Ende“ von X und „den Anfang“ von Y zugreifen müssen. Daher sind X und Y speziell codiert.



## Mit GOTO lassen sich Turingmaschinen simulieren (2)

- ◆ Es sei das Arbeitsalphabet einer Turingmaschine  $\Gamma = \{A, B, C, \#\}$  und wir nummerieren durch, also  $A = 1$  und  $\# = 4$ .
- ◆ Dann können wir ein Wort, das auch Buchstaben des Arbeitsalphabets besteht mit **p=5** (weil  $>4$ ) codieren (Gödelisieren)

$$ABAB\#\#AC \rightarrow 1 \cdot 5^7 + 2 \cdot 5^6 + 1 \cdot 5^5 + 2 \cdot 5^4 + 4 \cdot 5^3 + 4 \cdot 5^2 + 1 \cdot 5^1 + 3 \cdot 5^0$$

- ◆ Um nun an „das Ende“ zu gelangen, kann die so codierte Zahl einfach mod 5 gerechnet werden:

$$(1 \cdot 5^7 + 2 \cdot 5^6 + 1 \cdot 5^5 + 2 \cdot 5^4 + 4 \cdot 5^3 + 4 \cdot 5^2 + 1 \cdot 5^1 + 3 \cdot 5^0) \bmod 5 = 3 = C$$

- ◆ So gelangen wir an die Codierung von X, Y codieren wir spiegelverkehrt und gelangen so durch die mod Operation an „den Anfang“.



## Mit GOTO lassen sich Turingmaschinen simulieren (3)

M2:  $a := y \bmod p$

IF (z=1) AND (a=1) THEN GOTO M-1, 1

IF (z=1) AND (a=2) THEN GOTO M-1, 2

...

IF (z=k) AND (a=m) THEN GOTO M-k, m

...

M-9, 4:

$z := 7;$

$y = y \text{ DIV } B$

$y := b * y + 2$

$y := b * y + (x \bmod b)$

$x := x \bmod b$

$\sigma(z_9, a_4) = (z_7, a_2, L)$

neuer Zustand ist z7

erstes Zeichen von Y wird gelöscht

Rechtsshift und neues Zeichen (2)

Rechtsshift und neues Zeichen ist

„das Ende“ von X (Operation L)

Rechtsshift von X (Ende gelöscht)

$$\underbrace{a_{i_1} \dots a_{i_p}}_X \quad z_l \quad \underbrace{a_{j_1} \dots a_{j_q}}_Y$$

$Z = l$



- ◆ Jetzt fehlt uns noch, dass Turingmaschinen While-Schleifen ausführen können. Wie machen wir das?
- ◆ Wieso genügt es, dass Turingmaschinen nur *eine* While-Schleife simulieren müssen, um zu zeigen, dass

$$\text{LOOP} \subsetneq \text{WHILE} = \text{GOTO} = \text{TURING}$$



# Primitiv rekursive Funktionen

## ◆ Primitiv Rekursive (PR) Funktionen sind wie folgt definiert

1. Alle konstanten Funktionen sind Pr, wie z.B.  $f(n)=1$
2. Alle Projektionen sind Pr, wie z.B.  $f(a,b,c)=b$
3. Die Nachfolgefunktion  $s(n)=n+1$  ist Pr
4. Einsetzungen von Pr sind wieder Pr. Sind also  $f(n)$  und  $s(n)$  PR, dann ist auch  $f(g(n))$  Pr
5. Jede Funktion die durch primitive Rekursion entsteht ist Pr, wenn also mit  $g$  und  $h$  Pr folgendes Gleichungssystem aufgestellt werden kann:

$$\begin{aligned}f(0, \dots) &= g(\dots) \\f(n+1, \dots) &= h(f(n, \dots), \dots)\end{aligned}$$

## ◆ Pr = LOOP

$$\begin{aligned}add(0, x) &= x & mult(0, x) &= 0 \\add(n+1, x) &= s(add(n, x)) & mult(n+1, x) &= add(mult(n, x), x)\end{aligned}$$



# $\mu$ rekursive Funktionen

## ♦ $\mu$ -Rekursive (PR) Funktionen sind wie folgt definiert

1. Alle konstanten Funktionen sind  $\mu$ R, wie z.B.  $f(n)=1$
2. Alle Projektionen sind  $\mu$ R, wie z.B.  $f(a,b,c)=b$
3. Die Nachfolgefunktion  $s(n)=n+1$  ist  $\mu$ R
4. Einsetzungen von  $\mu$ R sind wieder  $\mu$ R. Sind also  $f(n)$  und  $s(n)$   $\mu$ R, dann ist auch  $f(g(n))$   $\mu$ R
5. Jede Funktion die durch primitive Rekursion entsteht ist  $\mu$ R, wenn also mit  $g$  und  $h$   $\mu$ R folgendes Gleichungssystem aufgestellt werden kann:

$$\begin{aligned} f(0, \dots) &= g(\dots) \\ f(n+1, \dots) &= h(f(n, \dots), \dots) \end{aligned}$$

6. Alle Funktionen, die durch den  $\mu$ -Operator entstehen, sind wieder  $\mu$ R

$$\mu g(x_1, x_2, \dots, x_n) = \begin{cases} \text{Die kleinste Zahl } y \\ \text{mit } g(x_1, \dots, x_n, y) = 0 & \text{falls } g(x_1, \dots, x_n, z) \text{ definiert ist} \\ & \text{für alle } z < y. \\ \text{undef} & \text{sonst} \end{cases}$$

$$f(x, y) = 4 \div (x + y)$$

| x \ y | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 |                |
|-------|---|---|---|---|---|---|---|---|----------------|
| 0     | 4 | 3 | 2 | 1 | 0 | 0 | 0 | 0 | $\mu f(0) = 4$ |
| 1     | 3 | 2 | 1 | 0 | 0 | 0 | 0 | 0 | $\mu f(1) = 3$ |
| 2     | 2 | 1 | 0 | 0 | 0 | 0 | 0 | 0 | $\mu f(2) = 2$ |
| 3     | 1 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | $\mu f(3) = 1$ |
| 4     | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | $\mu f(4) = 0$ |
| 5     | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | $\mu f(5) = 0$ |
| 6     | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | $\vdots$       |

## ♦ $\mu$ R = WHILE

<https://www.youtube.com/watch?v=-iPMZnYiUx4>



LOOP  
Primitiv-  
Rekursiv

Intuitiv berechenbar  
(Churchsche These)

$\mu$ Rekursiv  
WHILE  
Turing